

QUESTION 1

RANK EMPLOYEES BY SALARY WITHIN EACH DEPARTMENT

Problem Statement

TechNova Solutions wants to analyze employee compensation across departments.

The HR team needs a report showing how employees rank within their own department based on salary.

Employees with the same salary should receive the same rank.

The next lower salary should receive the next consecutive rank with no gaps.

Task

Write an SQL query joining the **employees** and **departments** tables that assigns each employee a rank within their department based on salary.

The ranking must:

- Restart for every department.
- Assign the same rank to employees with equal salaries.
- Use consecutive ranks without gaps.

Output Columns

The output must contain the following columns:

Column Name	Description
department_name	Department name
employee_name	Employee name
salary	Employee salary
salary_rank	Rank within department

Table Schema

Table: departments

Column Name	Data Type	Description
department_id	INT	Unique department id
department_name	VARCHAR(100)	Department name

Table: employees

Column Name	Data Type	Description
employee_id	INT	Unique employee id
employee_name	VARCHAR(100)	Employee name
department_id	INT	References departments(department_id)
salary	INT	Employee salary

Sample Input

Sample rows from departments

department_id	department_name
1	IT
2	HR
3	Finance

Sample rows from employees

employee_id	employee_name	department_id	salary
1	Alice	1	90000
2	Bob	1	90000
3	Charlie	1	85000
4	David	2	70000
5	Emma	2	65000

Expected Output Format

department_name employee_name salary salary_rank

Sample Output

department_name	employee_name	salary	salary_rank
HR	David	70000	1
HR	Emma	65000	2
IT	Alice	90000	1
IT	Bob	90000	1
IT	Charlie	85000	2

Explanation

IT Department

Employee	Salary
Alice	90000
Bob	90000
Charlie	85000

Ranking:

Employee	Rank
Alice	1
Bob	1
Charlie	2

Alice and Bob share Rank 1 because salaries are equal.

Charlie gets Rank 2 because it is the next distinct salary.

HR Department

Employee	Salary
David	70000
Emma	65000

Ranking:

Employee	Rank
David	1
Emma	2

Approach

Step 1: Join Employees and Departments

Join using:

e.department_id = d.department_id

This provides:

- department_name
 - employee_name
 - salary
-

Step 2: Apply Ranking

Use:

DENSE_RANK()

because:

- Same salary → same rank
- No gaps in ranking

Step 3: Restart Ranking

Use:

PARTITION BY department_name

Example:

IT → Rank starts from 1

HR → Rank starts from 1

Finance → Rank starts from 1

Step 4: Highest Salary Gets Rank 1

Use:

ORDER BY salary DESC

Step 5: Sort Final Output

Required order:

ORDER BY department_name, salary_rank, employee_name

SQL Query

```
SELECT  d.department_name, e.employee_name, e.salary,
        DENSE_RANK() OVER( PARTITION BY d.department_name    ORDER BY e.salary DESC
        ) AS salary_rank
FROM employees e
JOIN departments d
ON e.department_id = d.department_id
ORDER BY  d.department_name, salary_rank, e.employee_name;
```

Dry Run

IT Department

Before Ranking

Employee	Salary
Alice	90000
Bob	90000
Charlie	85000

After DENSE_RANK()

Employee	Rank
Alice	1
Bob	1
Charlie	2

Concepts Tested

- INNER JOIN
- Window Functions
- DENSE_RANK()
- PARTITION BY
- ORDER BY

```
SELECT  d.department_name,  e.employee_name,  e.salary,
        DENSE_RANK() OVER( PARTITION BY d.department_name      ORDER BY e.salary DESC
        ) AS salary_rank
FROM employees e
JOIN departments d
ON e.department_id = d.department_id
ORDER BY  d.department_name,  salary_rank,  e.employee_name;
```

QUESTION 2

RANK PRODUCTS BY SALES WITHIN EACH CATEGORY

Problem Statement

ShopKart is an online retail platform that sells products across multiple categories such as Electronics, Clothing, and Home Appliances.

The analytics team wants to identify the best-performing products within each category based on total sales.

Products with the same sales count should receive the same rank.

The next lower sales value should receive the next consecutive rank with no gaps.

Task

Write an SQL query joining the **products** and **categories** tables that assigns each product a rank within its category based on total sales.

The ranking must:

- Restart for every category.
- Assign the same rank to products with equal sales.
- Use consecutive ranks without gaps.

Output Columns

The output must contain the following columns:

Column Name	Description
category_name	Product category
product_name	Product name
total_sales	Total units sold
product_rank	Product rank within category

Table Schema**Table: categories**

Column Name	Data Type	Description
category_id	INT	Unique category identifier
category_name	VARCHAR(100)	Category name

Table: products

Column Name	Data Type	Description
product_id	INT	Unique product identifier
category_id	INT	References categories(category_id)
product_name	VARCHAR(100)	Product name
total_sales	INT	Total units sold

Sample Input**Sample rows from categories**

category_id	category_name
1	Electronics
2	Clothing
3	Furniture

Sample rows from products

product_id	category_id	product_name	total_sales
1	1	Laptop	950
2	1	Mobile	950
3	1	Tablet	900
4	2	Jeans	800
5	2	T-Shirt	700

Expected Output Format

category_name product_name total_sales product_rank

Sample Output

category_name	product_name	total_sales	product_rank
Clothing	Jeans	800	1
Clothing	T-Shirt	700	2
Electronics	Laptop	950	1
Electronics	Mobile	950	1
Electronics	Tablet	900	2

Explanation

Electronics Category

Product	Sales
Laptop	950
Mobile	950
Tablet	900

Ranking:

Product	Rank
Laptop	1
Mobile	1
Tablet	2

Laptop and Mobile share Rank 1 because both have identical sales.

Tablet receives Rank 2 because it has the next distinct sales value.

Clothing Category

Product	Sales
Jeans	800
T-Shirt	700

Ranking:

Product	Rank
Jeans	1
T-Shirt	2

Approach

Step 1: Join Products and Categories

Join:

`p.category_id = c.category_id`

This provides:

- category_name
 - product_name
 - total_sales
-

Step 2: Apply Ranking

Use:

`DENSE_RANK()`

because:

- Equal sales → same rank
 - No gaps in ranking
-

Step 3: Restart Ranking

Use:

PARTITION BY category_name

Example:

Electronics → Rank starts from 1

Clothing → Rank starts from 1

Furniture → Rank starts from 1

Step 4: Highest Sales Get Rank 1

Use:

ORDER BY total_sales DESC

Step 5: Sort Final Output

Required order:

ORDER BY category_name, product_rank, product_name

SQL Query

```
SELECT  c.category_name, p.product_name, p.total_sales,
        DENSE_RANK() OVER( PARTITION BY c.category_name ORDER BY p.total_sales DESC
        ) AS product_rank
FROM products p JOIN categories c
ON p.category_id = c.category_id
ORDER BY c.category_name, product_rank, p.product_name;
```

Dry Run

Electronics

Before Ranking

Product	Sales
Laptop	950
Mobile	950
Tablet	900

After Applying DENSE_RANK()

Product	Rank
Laptop	1
Mobile	1
Tablet	2

Clothing

Product	Sales	Rank
Jeans	800	1
T-Shirt	700	2

```
SELECT  c.category_name, p.product_name, p.total_sales,
        DENSE_RANK() OVER(PARTITION BY c.category_name ORDER BY p.total_sales DESC ) AS
product_rank
FROM products p JOIN categories c
ON p.category_id = c.category_id
ORDER BY c.category_name, product_rank, p.product_name;
```

QUESTION 3

RANK STUDENTS BY MARKS WITHIN EACH CLASS

Problem Statement

EduTrack Academy wants to analyze student performance across different classes.

The academic team needs a report showing how students rank within their own class based on marks obtained in the final examination.

Students with the same marks should receive the same rank.

The next lower mark should receive the next consecutive rank with no gaps.

Task

Write an SQL query joining the **students** and **classes** tables that assigns each student a rank within their class based on marks.

The ranking must:

- Restart for every class.
 - Assign the same rank to students with equal marks.
 - Use consecutive ranks without gaps.
-

Output Columns

The output must contain the following columns:

Column Name	Description
class_name	Name of the class
student_name	Name of the student
marks	Marks obtained
student_rank	Rank within the class

Table Schema

Table: classes

Column Name	Data Type	Description
class_id	INT	Unique class identifier
class_name	VARCHAR(100)	Class name

Table: students

Column Name	Data Type	Description
student_id	INT	Unique student identifier
class_id	INT	References classes(class_id)
student_name	VARCHAR(100)	Student name
marks	INT	Final examination marks

Sample Input

Sample rows from classes

class_id	class_name
1	Class A
2	Class B
3	Class C

Sample rows from students

student_id	class_id	student_name	marks
1	1	Alice	95
2	1	Bob	95
3	1	Charlie	90
4	2	David	88
5	2	Emma	82

Expected Output Format

class_name student_name marks student_rank

Sample Output

class_name	student_name	marks	student_rank
Class A	Alice	95	1
Class A	Bob	95	1
Class A	Charlie	90	2
Class B	David	88	1
Class B	Emma	82	2

Explanation**Class A**

Student	Marks
Alice	95
Bob	95
Charlie	90

Ranking:

Student	Rank
Alice	1
Bob	1
Charlie	2

Alice and Bob share Rank 1 because both scored 95 marks.

Charlie receives Rank 2 because 90 is the next distinct mark.

Class B

Student	Marks
David	88
Emma	82

Ranking:

Student	Rank
David	1
Emma	2

Approach

Step 1: Join Students and Classes

Join:

`s.class_id = c.class_id`

This gives access to:

- `class_name`
 - `student_name`
 - `marks`
-

Step 2: Apply Ranking

Use:

`DENSE_RANK()`

because:

- Equal marks receive the same rank.
 - No gaps in ranking.
-

Step 3: Restart Ranking for Every Class

Use:

`PARTITION BY class_name`

Example:

Class A → Rank starts from 1

Class B → Rank starts from 1

Class C → Rank starts from 1

Step 4: Highest Marks Get Rank 1

Use:

`ORDER BY marks DESC`

Step 5: Sort Final Output

Required order:

`ORDER BY class_name,
 student_rank,
 student_name`

Meaning:

1. Class ascending
 2. Rank ascending
 3. Student name ascending
-

SQL Query

```
SELECT  c.class_name,  s.student_name,  s.marks,  
        DENSE_RANK() OVER(PARTITION BY c.class_name  ORDER BY s.marks DESC ) AS student_rank  
FROM students s  
JOIN classes c  
ON s.class_id = c.class_id  
ORDER BY  c.class_name,  student_rank,  s.student_name;
```

Dry Run

Class A

Before Ranking

Student	Marks
Alice	95
Bob	95
Charlie	90

After Applying DENSE_RANK()

Student	Rank
Alice	1
Bob	1
Charlie	2

Class B

Before Ranking

Student	Marks
David	88
Emma	82

After Applying DENSE_RANK()

Student	Rank
David	1
Emma	2

SELECT

```
c.class_name,  
s.student_name,  
s.marks,  
DENSE_RANK() OVER(  
    PARTITION BY c.class_name  
    ORDER BY s.marks DESC  
) AS student_rank
```

FROM students s

JOIN classes c

ON s.class_id = c.class_id

ORDER BY

```
c.class_name,  
student_rank,  
s.student_name;
```

QUESTION 4 – RANK MOVIES BY RATING WITHIN EACH GENRE

Output Columns

| genre | movie_title | rating | movie_rank |

Tables

genres

| genre_id | genre |

movies

| movie_id | genre_id | movie_title | rating |

Sample Data

genres

genre_id	genre
1	Action
2	Comedy

movies

movie_id	genre_id	movie_title	rating
1	1	Fast Strike	9.5
2	1	Thunder Run	9.5
3	1	Battle Zone	8.8
4	2	Funny Day	8.9
5	2	Happy Hour	8.4

Sample Output

genre	movie_title	rating	movie_rank
Action	Fast Strike	9.5	1
Action	Thunder Run	9.5	1
Action	Battle Zone	8.8	2
Comedy	Funny Day	8.9	1
Comedy	Happy Hour	8.4	2

SQL

SELECT

```
    g.genre,  
    m.movie_title,  
    m.rating,  
    DENSE_RANK() OVER(  
        PARTITION BY g.genre  
        ORDER BY m.rating DESC  
    ) AS movie_rank
```

FROM movies m

JOIN genres g

ON m.genre_id = g.genre_id

ORDER BY g.genre,movie_rank,m.movie_title;

QUESTION 5 – RANK SALES REPRESENTATIVES BY SALES WITHIN EACH REGION

Output Columns

| region | rep_name | total_sales | sales_rank |

Tables

regions

| region_id | region |

sales_reps

| rep_id | region_id | rep_name | total_sales |

Sample Output

region	rep_name	total_sales	sales_rank
East	John	90000	1
East	Peter	90000	1
East	Sam	85000	2

SQL

```
SELECT
    r.region,
    s.rep_name,
    s.total_sales,
    DENSE_RANK() OVER(
        PARTITION BY r.region
        ORDER BY s.total_sales DESC
    ) AS sales_rank
FROM sales_reps s
JOIN regions r
ON s.region_id=r.region_id
ORDER BY r.region,sales_rank,s.rep_name;
```

QUESTION 6 – RANK COURSES BY ENROLLMENT WITHIN EACH DEPARTMENT

Output Columns

| department_name | course_name | enrollment_count | course_rank |

Tables

departments

| department_id | department_name |

courses

| course_id | department_id | course_name | enrollment_count |

SQL

```
SELECT
    d.department_name, c.course_name, c.enrollment_count,
    DENSE_RANK() OVER( PARTITION BY d.department_name ORDER BY c.enrollment_count DESC
    ) AS course_rank
FROM courses c
JOIN departments d
ON c.department_id=d.department_id
ORDER BY d.department_name,course_rank,c.course_name;
```

QUESTION 7 – RANK HOTELS BY CUSTOMER RATING WITHIN EACH CITY

Output Columns

| city | hotel_name | rating | hotel_rank |

Tables

hotels

| hotel_id | city | hotel_name | rating |

Sample Data

hotel_name	city	rating
Grand Palace	Mumbai	4.8
Royal Stay	Mumbai	4.8
Ocean View	Goa	4.9

SQL

```
SELECT
    city,
    hotel_name,
    rating,
    DENSE_RANK() OVER(
        PARTITION BY city
        ORDER BY rating DESC
    ) AS hotel_rank
FROM hotels
ORDER BY city,hotel_rank,hotel_name;
```

QUESTION 8 – RANK FLIGHTS BY REVENUE WITHIN EACH COUNTRY

Output Columns

| country | flight_number | revenue | revenue_rank |

Tables

countries

| country_id | country |

flights

| flight_id | country_id | flight_number | revenue |

SQL

```
SELECT
    c.country,
    f.flight_number,
    f.revenue,
    DENSE_RANK() OVER(
        PARTITION BY c.country
        ORDER BY f.revenue DESC
    ) AS revenue_rank
FROM flights f
JOIN countries c
ON f.country_id=c.country_id
ORDER BY c.country,revenue_rank,f.flight_number;
```

QUESTION 9 – RANK BOOKS BY SALES WITHIN EACH CATEGORY

Output Columns

| category | book_title | copies_sold | book_rank |

Tables

categories

| category_id | category |

books

| book_id | category_id | book_title | copies_sold |

SQL

```
SELECT
    c.category,
    b.book_title,
    b.copies_sold,
    DENSE_RANK() OVER(
        PARTITION BY c.category
        ORDER BY b.copies_sold DESC
    ) AS book_rank
FROM books b
JOIN categories c
ON b.category_id=c.category_id
ORDER BY c.category,book_rank,b.book_title;
```

QUESTION 10 – RANK HOSPITALS BY PATIENT COUNT WITHIN EACH STATE

Output Columns

| state | hospital_name | patient_count | hospital_rank |

Tables

states

| state_id | state |

hospitals

| hospital_id | state_id | hospital_name | patient_count |

Sample Data

hospital_name	state	patient_count
City Care	Maharashtra	1500
Life Hospital	Maharashtra	1500
Apollo Health	Karnataka	1400

SQL

```
SELECT    s.state,    h.hospital_name,    h.patient_count,
    DENSE_RANK() OVER(
        PARTITION BY s.state
        ORDER BY h.patient_count DESC
    ) AS hospital_rank
FROM hospitals h
JOIN states s
ON h.state_id=s.state_id
ORDER BY s.state,hospital_rank,h.hospital_name;
```
